

Intuitive Verification of Sequential Programs Using Hybrid Reasoning

Shushu Wu, Xiwei Wu, Chengxi Yang, and Qinxiang Cao^{*}

Shanghai Jiao Tong University, Shanghai, China
{Ciel77,yashen,arcadia-y,caoqinxiang}@sjtu.edu.cn

Abstract. Verifying the functional correctness of real-world code with complex algorithms requires reasoning about both implementation correctness and algorithm correctness. Implementation correctness, which relates concrete programs to abstract functional models, is inherently relational, yet is traditionally formalized using standard Hoare logic alone. This paper proposes a hybrid reasoning approach that combines relational Hoare logic and standard Hoare logic. Our key insight is that verifying implementation correctness naturally calls for relational Hoare logic, while algorithm correctness is well-suited to standard Hoare logic. Compared to traditional verifications that rely exclusively on standard Hoare logic, our approach simplifies proofs and offers more intuitive reasoning patterns. We demonstrate this in a setting where the concrete language is a C-like imperative language and the abstract algorithm is written in a simple nondeterministic functional language. Through case studies on binary search trees and merge sort, we show how our hybrid reasoning approach addresses challenges in verifying complex algorithms, reduces proof complexity, and enhances clarity.

Keywords: Program verification · Relational Hoare logic · Separation logic

1 Introduction

Proving the functional correctness of real-world code that incorporates complex algorithms is challenging [10,22,31,40,11]. A commonly adopted strategy is the two-layer method [3], which separates proofs into two layers. Implementation correctness ensures that the concrete program correctly implements a functional model. Algorithm correctness validates that the functional model meets its specification.

However, proving implementation correctness is often difficult in practice. Most existing work relies on standard Hoare logic, which often leads to verbose proofs or unnatural loop invariants. Consider, for example, verifying a binary search tree (BST) insertion function, as shown in Fig. 1. Its functional correctness requires that after insertion, the resulting tree remains ordered and stores

^{*} Corresponding Author

<pre> void bst_ins(int x, int v, struct tree ** p) { struct tree *q = *p; while (q && x != q->key) { if (x < q->key) p = &q->left; else p = &q->right; q = *p; } if (!q) { q = malloc(sizeof(struct tree)); q->key = x; q->value = v; q->left = q->right = NULL; *p = q; } else { q->value = v; } } </pre>	<pre> function insert(x, v, s): match s with leaf ⇒ node(leaf, x, v, leaf) node(a, y, w, b) ⇒ if x < y then let a' = insert(x, v, a) in node(a', y, w, b) else if x > y then let b' = insert(x, v, b) in node(a, y, w, b') else node(a, x, v, b) end </pre>
---	---

Fig. 1: BST insertion: concrete C implementation vs. abstract model. **leaf** denotes the empty tree and **node**(l, k, v, r) denotes a node with left child l , key k , value v , and right child r .

the new key-value pair. To prove this, the two-layer method introduces a functional model **insert** [2], as shown on the right of Fig. 1, and separates the proof into two parts. Algorithm correctness validates that **insert** returns a tree whose left subtree keys are smaller than the root and right subtree keys are larger, and that the new key-value pair is correctly stored. Implementation correctness shows the C program faithfully implements **insert**, and is formalized as:

$$\{\text{storetree}(p_0, t_0)\} \text{bst_ins}(k, v, p_0) \{\text{storetree}(p_0, \text{insert}(k, v, t_0))\} \quad (1)$$

That is, if the memory initially stores a tree t_0 at p_0 , then after execution it stores the result of **insert**(k, v, t_0). However, proving this triple is tedious. To prove this triple, Cao et al. [10] employ the following loop invariant:

$$\exists t P. P(\text{insert}(k, v, t)) = \text{insert}(k, v, t_0) \wedge \text{storetree}(p, t) * \text{storezipper}(p_0, p, P)$$

This invariant states the storage of a subtree t and its contextual zipper [18] P (a path from the root to t). Importantly, it requires that combining local updates to the currently focused subtree t with the context P is equivalent to global updates to the original tree t_0 . Requiring this equivalence is counterintuitive: the concrete program modifies only the local pointer p , yet the proof must consider both local changes of the subtree t and global changes of the entire tree t_0 .

This difficulty arises because implementation correctness is inherently relational: it relates a concrete program to an abstract algorithm. Standard Hoare logic, however, can only reason about a single program. The loop invariant cannot directly describe the relationship between the two programs at each intermediate step. Instead, it must encode the abstract program's behavior through value

<pre> struct list * mergesort(struct list * x) { int l = listlen(x); if (l < 8) return inssort(x); else { struct list * p, * q; p = NULL; q = NULL; split(x, &p, &q); p = mergesort(p); q = mergesort(q); return merge(p, q); } } </pre>	<pre> function extsort_A(x): return r such that permutation(x, r) ∧ sorted(r) function mergesort_A(x): choose extsort_A(x) or require length(x) ≥ 2; let (p₁, q₁) = split_A(x, nil, nil) in let p₂ = mergesort_A(p₁) in let q₂ = mergesort_A(q₁) in merge_A(p₂, q₂) end </pre>
---	---

Fig. 2: Hybrid sorting: concrete C implementation vs. abstract model. The abstract model nondeterministically chooses between an external sorter `extsortA` and a recursive merge sort.

equalities. In the invariant above, $P(\text{insert}(k, v, t)) = \text{insert}(k, v, t_0)$ serves exactly this purpose: it expresses what the abstract insertion would produce at the current point by relating it to the final result $\text{insert}(k, v, t_0)$. This indirection is what makes the invariant unnatural. Relational Hoare logic, which reasons about two programs simultaneously, is a natural fit for such proofs. Besides, relational verification is often simpler than functional verification, particularly when the two programs are similar [6].

However, some refinement proofs naturally reduce to standard functional correctness arguments. Relational proofs can require standard Hoare logic. To illustrate, consider the hybrid sorting algorithm in Fig. 2. The C program uses insertion sort for small inputs (fewer than 8 elements) and merge sort for larger ones. In practice, different implementations may use different thresholds for switching strategies and different algorithms for small inputs. To capture this family of implementations, the abstract model uses a nondeterministic choice between two branches: a recursive merge sort that splits, recursively sorts, and merges, and an external sorter `extsortA` that is a black box specifying only that the output is a sorted permutation of the input. Verifying that the concrete program implements this abstract model requires both relational and standard Hoare logic. For the merge sort part, concrete subprograms (e.g., `split(x, &p, &q)`) align with their abstract counterparts (e.g., `splitA(x, nil, nil)`), so relational Hoare logic can decompose the proof into small refinement obligations. For the insertion sort part, `inssort(x)` must refine `extsortA(x)`. Since `extsortA` only requires the output to be a sorted permutation, this reduces to proving the functional correctness of `inssort(x)`, which is naturally handled by standard Hoare logic.

Based on these observations, we propose a hybrid reasoning approach that explicitly combines relational Hoare logic [7,43] and standard Hoare logic [17,32]. We use relational Hoare logic for implementation correctness, enabling more intuitive reasoning patterns and simplifying previously tedious proofs. Relational Hoare logic proofs in turn rely on standard Hoare logic when refinement obligations reduce to functional correctness of individual subprograms. Algorithm correctness continues to rely on standard Hoare logic, and by composing the two layers, the overall functional correctness can be derived.

Our main contributions are as follows:

- We reveal that certain standard Hoare logic proofs for implementation correctness are in essence relational. Formalizing these proofs using relational Hoare logic enables more natural verification and simplifies the previously tedious proofs.
- We propose a hybrid reasoning approach that combines relational Hoare logic and standard Hoare logic. Specifically, we use relational Hoare triples to formalize implementation correctness and standard Hoare triples to formalize algorithm correctness. Furthermore, while proving relational triples, we sometimes employ standard Hoare logic for subprograms.
- We instantiate our approach in a framework where the concrete language is a C-like imperative language and the abstract programs are written in a simple nondeterministic functional language.
- We validate our approach through case studies on BSTs and merge sort. All results are mechanized in Rocq [42].

2 Preliminary: Relational Hoare Logic

Program-as-resource assertions [37] originate from concurrent program verification [27,26,34,15,33,36], where they embed the pending operations of a thread directly into invariants or protocols to reason about inter-thread cooperation. Concretely, a program-as-resource assertion $\mathcal{P}$ is a predicate over the concrete state, the abstract state, and the abstract program itself. The main judgment for refinement is a relational Hoare triple of the form:

$$\{[c^{\text{abs}}] \wedge \mathcal{P}\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge \mathcal{Q}(r^{\text{con}}, r^{\text{abs}})\}$$

where r^{con} and r^{abs} denote the concrete and abstract return values, respectively. Informally, the triple asserts that for any execution of c^{con} from an initial configuration satisfying $\mathcal{P}$, there exists a corresponding execution of c^{abs} such that the return values r^{con} and r^{abs} together satisfy $\mathcal{Q}$. For example, the implementation correctness of BST insertion (Fig. 1) can be expressed as:

$$\begin{aligned} & \{[\text{insert}(k, v, t_0)] \wedge \text{storetree}(\mathbf{p}_0, t_0)\} \\ & \quad \text{bst_ins}(k, v, \mathbf{p}_0) \\ & \{ \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge \text{storetree}(\mathbf{p}_0, r^{\text{abs}}) \} \end{aligned} \tag{2}$$

Here $\text{storetree}(p_0, t_0)$ asserts that the memory stores tree t_0 at address p_0 . The precondition embeds $\text{insert}(k, v, t_0)$ as the abstract program to be refined, and the postcondition asserts termination with result r^{abs} (via **ret** r^{abs}) now stored at p_0 . Since the concrete insertion modifies the tree in place and returns no value, r^{con} is omitted. We prove this triple in Sec. 3.1.

We now formalize the relational Hoare triple in general.

Definition 1 (Relational Hoare Triples). *Triple $\{\mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}$ is valid if, given any initial states $\sigma_1^{\text{con}}, \sigma_1^{\text{abs}}$ and abstract program c_1^{abs} satisfying $\mathcal{P}$, for every $(\sigma_2^{\text{con}}, r^{\text{con}})$ reachable by executing c^{con} from σ_1^{con} , there exist σ_2^{abs} and c_2^{abs} such that*

$$(\sigma_1^{\text{abs}}, c_1^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, c_2^{\text{abs}}) \quad \text{and} \quad (\sigma_2^{\text{con}}, \sigma_2^{\text{abs}}, c_2^{\text{abs}}) \text{ satisfies } \mathcal{Q}(r^{\text{con}}).$$

Here $(\sigma_1^{\text{abs}}, c_1^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, c_2^{\text{abs}})$ is a reduction relation, which describes how c_1^{abs} reduces under state σ_1^{abs} to a new program c_2^{abs} and (possibly) updated state σ_2^{abs} .

In this paper, concrete programs are written in a C-like imperative language and abstract programs in a simple nondeterministic functional language (Fig. 1, Fig. 2). Inference rules are introduced as needed in subsequent sections.

3 An Insight: The Relational Nature of Standard Hoare Logic Proofs

In this section, we demonstrate that implementation correctness is relational in nature, and that relational Hoare logic yields more intuitive proofs (Sec. 3.1). We then compose the relational proof with a standard algorithm correctness proof to derive full functional correctness (Sec. 3.2).

3.1 Implementation Correctness as Relational Hoare Triples

As shown in Sec. 2, the implementation correctness of BST insertion can be expressed as the relational triple (2), which states that the concrete program $\text{bst_ins}(k, v, p_0)$ (pointer manipulation in loops) and the abstract program $\text{insert}(k, v, t_0)$ (recursive tree operations) produce consistent results. We now show how relational Hoare logic yields a more intuitive proof.

Fig. 3 presents the relational inference rules used in this proof. REL-WH¹ handles while loops with a relational invariant $\mathcal{P}$. REL-SEQ decomposes a sequential concrete program $c_1^{\text{con}}; c_2^{\text{con}}$ into two relational subgoals, connected by an intermediate assertion $\mathcal{R}$. CON-STORE handles memory stores, updating the heap while preserving the rest of the assertion via separating conjunction. ABS-PURE rewrites the abstract program via a pure reduction $\rightarrow_p$ (function unfolding, let-in reassociation, etc.).

¹ The REL-WH rule assumes the loop body contains no **return** or **break** statements; that is, every execution of the body either continues to the next iteration or exits via the loop condition.

$$\begin{array}{c}
\text{REL-WH} \\
\frac{\{\mathcal{P} \wedge b^{\text{con}}\} \ c^{\text{con}} \ \{\mathcal{P}\}}{\{\mathcal{P}\} \ \text{while } b^{\text{con}} \ \text{do } c^{\text{con}} \ \{\mathcal{P} \wedge \neg b^{\text{con}}\}} \\
\\
\text{ABS-PURE} \\
\frac{c_1^{\text{abs}} \rightarrow_p c_2^{\text{abs}} \quad \{[c_2^{\text{abs}}] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}{\{[c_1^{\text{abs}}] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}} \\
\\
\text{REL-SEQ} \qquad \text{CON-STORE} \\
\frac{\{\mathcal{P}\} \ c_1^{\text{con}} \ \{\mathcal{R}\} \quad \{\mathcal{R}\} \ c_2^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}{\{\mathcal{P}\} \ c_1^{\text{con}}; c_2^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}} \qquad \frac{}{\{\mathbf{p} \mapsto _ * \mathcal{F}\} \ \text{store}(\mathbf{p}, e) \ \{\mathbf{p} \mapsto e * \mathcal{F}\}}
\end{array}$$

Fig. 3: An excerpt of relational inference rules.

Since the concrete program primarily consists of a **while** loop, we apply REL-WH to reduce the proof to finding a loop invariant. Because the invariant is a program-as-resource assertion, it must describe both the current concrete memory and the remaining abstract computation at each iteration. The concrete memory can be partitioned into two disjoint segments: the subtree t rooted at the current focus $\mathbf{p}$, which corresponds to the tree manipulated by the abstract program's recursive calls, and a zipper context P [18] that records the path from the root $\mathbf{p}_0$ to $\mathbf{p}$. Intuitively, P is a tree with a hole: the hole marks where the focused subtree belongs. At each iteration, the remaining abstract computation is: first perform the recursive insertion on t , then fill the hole in P with the result. Consider the example in Fig. 4, where each node stores a key-value pair (we write w_0 for the value at y_0 , and w_1 for the value at y_1). After the first iteration ($k < y_0$), $P = \text{node}(\square, y_0, w_0, b_0)$, where $\square$ denotes the hole. After the second ($k > y_1$), P grows to $\text{node}(\text{node}(a_1, y_1, w_1, \square), y_0, w_0, b_0)$. In general, P is the composition of such one-hole contexts along the entire path from the root to the current focus. Using the separating conjunction $*$ [32] to assert disjointness, we obtain the following loop invariant:

$$\exists t, P. [\text{let } x = \text{insert}(k, v, t) \text{ in } P(x)] \wedge \text{storetree}(\mathbf{p}, t) * \text{storezipper}(\mathbf{p}_0, \mathbf{p}, P)$$

Next, we need to prove that the loop body preserves this invariant. Consider the case when k is greater than the key y_1 of the root of the focused subtree t , such as in Fig. 4. Suppose $t = \text{node}(a_1, y_1, w_1, b)$, where a_1 and b are the left and right subtrees. With rule ABS-PURE, we can transform the abstract program:

Proposition 1. *When $k > y_1$ and $t = \text{node}(a_1, y_1, w_1, b)$:*

$$\begin{aligned}
& \text{let } x = \text{insert}(k, v, t) \text{ in } P(x) \\
& \rightarrow_p \text{let } x = (\text{let } r = \text{insert}(k, v, b) \text{ in } \text{node}(a_1, y_1, w_1, r)) \text{ in } P(x) \\
& \rightarrow_p \text{let } r = \text{insert}(k, v, b) \text{ in let } x = \text{node}(a_1, y_1, w_1, r) \text{ in } P(x) \\
& \rightarrow_p \text{let } r = \text{insert}(k, v, b) \text{ in } P(\text{node}(a_1, y_1, w_1, r))
\end{aligned}$$

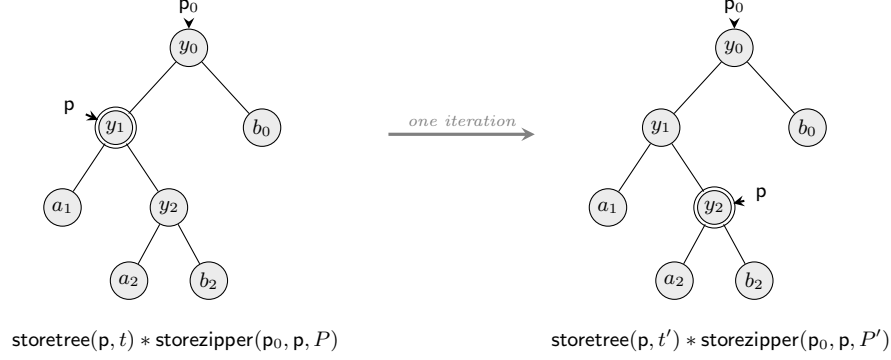


Fig. 4: Zipper decomposition during BST insertion ($k < y_0$ and $k > y_1$). The double-circled node is the focus pointed to by p . One iteration advances p from y_1 to y_2 : the focused subtree shrinks and the zipper context grows.

After this reduction, the abstract program becomes:

let $r = \text{insert}(k, v, b)$ **in** $P(\text{node}(a_1, y_1, w_1, r))$

For the concrete side, we apply REL-SEQ and CON-STORE to evaluate the loop body, which advances pointer p from y_1 to y_2 in Fig. 4. It remains to show that combining the root node (y_1, w_1) , the subtree a_1 , and the old context P yields the updated zipper context, which is straightforward.

In contrast, the traditional two-layer method formalizes implementation correctness as the standard Hoare triple (1), whose loop invariant is:

$$\exists t P. P(\text{insert}(k, v, t)) = \text{insert}(k, v, t_0) \wedge \text{storetree}(p, t) * \text{storezipper}(p_0, p, P)$$

Both invariants require the consistent representation of the subtree and zipper context. However, in the relational proof, there is no need to explicitly reason that combining the context P with the updated subtree $\text{insert}(k, v, t)$ yields the overall result $\text{insert}(k, v, t_0)$. Instead, this equivalence is implicitly decomposed by the abstract program transformations in the relational Hoare triple.

This observation simplifies a key part of the reasoning: the loop invariant. In program verification, identifying appropriate invariants is often the most difficult step. Our relational formulation suggests invariants that align naturally with the structure of the implementation and its abstract specification, reducing the proof burden. Thus, we argue that implementation correctness is best understood as a relational property, and that employing relational Hoare triples brings more intuitive loop invariants and less complex proofs.

3.2 Composing Program Refinement with Algorithm Correctness

Since the relational triple shows that the concrete program refines the abstract model, correctness properties proven for the abstract program (e.g., preservation of the BST property) transfer directly to the concrete implementation.

Concretely, for the BST insertion example, we use the following standard Hoare triple to formalize the correctness of the abstract program:

$$\{\text{ordered}(t_0) \wedge t_0 \sim m\} \text{ insert}(k, v, t_0) \{r^{\text{abs}}. \text{ordered}(r^{\text{abs}}) \wedge r^{\text{abs}} \sim m[k \mapsto v]\}$$

Here $\text{ordered}(t)$ asserts that t is an ordered tree: every node's key is greater than all keys in its left subtree and smaller than all keys in its right subtree. $t \sim m$ asserts that the key-value pairs stored in t correspond to the finite map m . This triple states that inserting (k, v) into a valid BST t_0 yields a valid BST r^{abs} that corresponds to the updated map $m[k \mapsto v]$.

We can then compose this triple with the relational triple (2) to derive the functional correctness of the concrete program. Here $\text{storemap}(p_0, m)$ denotes that there exists a valid BST t_0 stored at p_0 with key-value pairs corresponding to a mapping m :

$$\{\text{storemap}(p_0, m)\} \text{ bst_ins}(k, v, p_0) \{\text{storemap}(p_0, m[k \mapsto v])\} \quad (3)$$

In this paper, we refer to this composition as *vertical composition*. Our relational Hoare logic provides the following vertical composition rule:

$$\frac{\begin{array}{c} \{[c^{\text{abs}}] \wedge \mathcal{P}\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge Q(r^{\text{con}}, r^{\text{abs}})\} \\ \{P\} c^{\text{abs}} \{r^{\text{abs}}. Q(r^{\text{abs}})\} \end{array}}{\{\mathcal{P} \odot P\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. Q(r^{\text{con}}, r^{\text{abs}}) \odot Q(r^{\text{abs}})\}}$$

Here $\mathcal{P}$, Q are program-as-resource assertions and P , Q are assertions over the abstract state. The operator $\odot$ projects to a concrete-state predicate: $\mathcal{P} \odot P$ holds for σ^{con} when there exists an abstract state σ^{abs} such that $\mathcal{P}$ holds for $(\sigma^{\text{con}}, \sigma^{\text{abs}})$ and P holds for σ^{abs} . In the BST example, $\mathcal{P}$ is $\text{storetree}(p_0, t_0)$ and P is $\text{ordered}(t_0) \wedge t_0 \sim m$, so $\mathcal{P} \odot P$ is equivalent to $\text{storemap}(p_0, m)$, i.e., $\exists t_0. \text{storetree}(p_0, t_0) \wedge \text{ordered}(t_0) \wedge t_0 \sim m$. Composing the relational triple (2) with the above algorithm correctness triple yields (3).

4 Hybrid Reasoning with Relational and Standard Hoare Logics

In this section, we present our hybrid reasoning approach that combines relational and standard Hoare logic to verify programs with function calls. We give a high-level overview in Sec. 4.1 and formalize the language and logic in Sec. 4.2.

4.1 The Hybrid Reasoning Approach

As illustrated in Sec. 1, the hybrid sorting example (Fig. 2) involves concrete function calls that require different proof strategies. Some calls, such as `split` and `merge`, align with abstract counterparts and are best verified relationally. Others, such as `inssort`, reduce to standalone functional correctness and are best handled by standard Hoare logic. In particular, the relational proof of

`mergesort` invokes the standard Hoare logic proof of `inssort`. Conversely, using the two-layer method to prove functional correctness requires implementation correctness, which relies on relational Hoare logic. To support this interplay, we augment both standard and relational Hoare triples with proof contexts that record verified function specifications, allowing each proof to reuse results established by either approach. Concretely, a standard proof context S_V collects function specifications

$$(f : \Pi (\vec{a} : A). \{P(\vec{a})\} \{r^{\text{con}}. Q(\vec{a}, r^{\text{con}})\})$$

Here $\Pi (\vec{a} : A)$ binds a vector $\vec{a}$ of typed parameters, which may include the function's arguments as well as logical variables that relate the precondition P and the postcondition Q . The specification thus states that, for any such $\vec{a}$, calling f from a state satisfying $P(\vec{a})$ yields a return value and state satisfying $Q(\vec{a}, r^{\text{con}})$. A relational proof context $S_{V\exists}$ collects relational specifications

$$(f : \Pi (\vec{a} : A). \{[c^{\text{abs}}] \wedge P(\vec{a})\} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge Q(\vec{a}, r^{\text{con}}, r^{\text{abs}})\})$$

stating that calling f refines an abstract program c^{abs} . The two primary contextual judgments are:

$$S_V, S_{V\exists} \vdash \{P\} c^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\} \quad S_V, S_{V\exists} \vdash \{P\} c^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\}$$

Both are natural extensions of their context-free counterparts: the triple holds for any environment that validates both S_V and $S_{V\exists}$. Formal definitions are given in Sec. 4.2.

Using these judgments, we prove the functional correctness of a concrete program c^{con} that refines an abstract algorithm c^{abs} . The interplay between the two logics works in both directions:

Relational Hoare logic for standard Hoare logic Decompose the proof into implementation correctness and algorithm correctness.

- *Implementation correctness via relational Hoare logic:* We formalize that the concrete program c^{con} refines the abstract algorithm c^{abs} using a relational Hoare triple

$$\{[c^{\text{abs}}] \wedge P\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge Q(r^{\text{con}}, r^{\text{abs}})\}.$$

- *Algorithm correctness via standard Hoare logic:* We prove that c^{abs} satisfies its specification via a standard Hoare triple

$$\{P\} c^{\text{abs}} \{r^{\text{abs}}. Q(r^{\text{abs}})\}$$

where P and Q are predicates over abstract states.

- *Composition:* Applying the vertical composition rule to compose the implementation correctness and algorithm correctness, we obtain a correctness triple for the concrete program c^{con} .

$$\frac{\text{VERTICAL-COMP} \quad \begin{array}{c} S_V, S_{V\exists} \vdash \{[c^{\text{abs}}] \wedge P\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge Q(r^{\text{con}}, r^{\text{abs}})\} \\ \{P\} c^{\text{abs}} \{r^{\text{abs}}. Q(r^{\text{abs}})\} \end{array}}{S_V \vdash \{P \odot P\} c^{\text{con}} \{r^{\text{con}}. \exists r^{\text{abs}}. Q(r^{\text{con}}, r^{\text{abs}}) \odot Q(r^{\text{abs}})\}}$$

Standard Hoare logic for relational Hoare logic Conversely, relational proofs may reduce to standard Hoare logic proofs when part of the concrete program does not interact with the abstract side. The CON-FOCUS rule supports this by decomposing $c_1^{\text{con}}; c_2^{\text{con}}$ and verifying c_1^{con} via a contextual standard Hoare triple, so that function specifications from the proof contexts can be used:

$$\text{CON-FOCUS} \quad \frac{S_{\forall}, S_{\forall\exists} \vdash \{P^{\text{con}}\} c_1^{\text{con}} \{R^{\text{con}}\} \quad S_{\forall}, S_{\forall\exists} \vdash \{\mathcal{F} \wedge R^{\text{con}}\} c_2^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\}}{S_{\forall}, S_{\forall\exists} \vdash \{\mathcal{F} \wedge P^{\text{con}}\} c_1^{\text{con}}; c_2^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\}}$$

Here $\mathcal{F}$ is any assertion independent of the concrete state, so that it is preserved by the execution of c_1^{con} . In Sec. 3, individual concrete statements (e.g., memory stores) are handled by basic relational rules such as CON-STORE. These rules suffice when the concrete side involves primitive operations. For function calls with a clear abstract counterpart, the proof reduces to a sub-refinement goal. However, in the merge sort example, the call to `insort` reduces to proving its standalone functional correctness, which is naturally expressed as a standard Hoare triple. The CON-FOCUS rule enables this by delegating c_1^{con} to standard Hoare logic with access to the proof contexts. Note that the rules in Fig. 3 remain valid when augmented with proof contexts.

4.2 Formalization

Concrete language. Concrete programs are written in a C-like imperative language Imp with standard constructs (assignment, sequencing, conditionals, loops, and function calls). The behavior of a program $c^{\text{con}} \in \text{Imp}$ is captured by a ternary relation: $(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}})$ means that executing c^{con} from state σ_1^{con} may terminate at state σ_2^{con} with return status $r^{\text{con}} \in \text{Val}_{\perp}$. Here $\text{Val}_{\perp}$ is the lifted value type: $r^{\text{con}} = \perp$ when execution encounters no return statement, and $r^{\text{con}} = v$ when a return statement yields value v .

To reason about function calls, we parameterize the semantics with an environment $\chi : \text{Fn} \rightarrow \mathcal{P}(\Sigma^{\text{con}} \times \text{Val} \times \Sigma^{\text{con}})$, mapping each function name to its input-output behavior. We write $\llbracket c^{\text{con}} \rrbracket^{\chi}$ for the denotation of c^{con} under environment χ . For a sequencing composition $c_1^{\text{con}}; c_2^{\text{con}}$, when c_1^{con} encounters no return ($r^{\text{con}} = \perp$), execution continues with c_2^{con} ; when c_1^{con} returns a value, the result propagates directly and c_2^{con} is skipped.

$$(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \llbracket c_1^{\text{con}}; c_2^{\text{con}} \rrbracket^{\chi} \text{ if } \begin{cases} (\sigma_1^{\text{con}}, \perp, \sigma_0^{\text{con}}) \in \llbracket c_1^{\text{con}} \rrbracket^{\chi} \text{ and } (\sigma_0^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \llbracket c_2^{\text{con}} \rrbracket^{\chi} \\ (\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \llbracket c_1^{\text{con}} \rrbracket^{\chi} \text{ and } r^{\text{con}} \neq \perp \end{cases}$$

For function calls, we distinguish two forms:

$$\begin{aligned} (\sigma_1^{\text{con}}, \perp, \sigma_2^{\text{con}}) &\in \llbracket \text{call } f \rrbracket^{\chi} && \text{if } (\sigma_1^{\text{con}}, v, \sigma_2^{\text{con}}) \in \chi(f) \\ (\sigma_1^{\text{con}}, \perp, \sigma_2^{\text{con}}[x \mapsto v]) &\in \llbracket x := \text{call } f \rrbracket^{\chi} && \text{if } (\sigma_1^{\text{con}}, v, \sigma_2^{\text{con}}) \in \chi(f) \end{aligned}$$

A bare call `call` f discards the return value, while $x := \text{call } f$ stores it in x . Neither form triggers a return in the caller.

Abstract language. Abstract programs are nondeterministic specifications written in a simple functional language (Fig. 1, Fig. 2). An abstract program c^{abs} of return type A over state space Σ^{abs} is given a denotational semantics $\llbracket c^{\text{abs}} \rrbracket \subseteq \Sigma^{\text{abs}} \times A \times \Sigma^{\text{abs}}$, where $(\sigma_1^{\text{abs}}, a, \sigma_2^{\text{abs}}) \in \llbracket c^{\text{abs}} \rrbracket$ means that from state σ_1^{abs} , the program may return a and transition to σ_2^{abs} . For example, $\llbracket \text{ret } a \rrbracket = \{(\sigma^{\text{abs}}, a, \sigma^{\text{abs}}) \mid \sigma^{\text{abs}} \in \Sigma^{\text{abs}}\}$. We define the reduction relation $(\sigma_1^{\text{abs}}, c_1^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, c_2^{\text{abs}})$ as follows:

$$\begin{aligned} (\sigma_1^{\text{abs}}, c_1^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, c_2^{\text{abs}}) \text{ holds if,} \\ \text{for any } a \text{ and } \sigma_3^{\text{abs}}, (\sigma_2^{\text{abs}}, a, \sigma_3^{\text{abs}}) \in \llbracket c_2^{\text{abs}} \rrbracket \text{ implies } (\sigma_1^{\text{abs}}, a, \sigma_3^{\text{abs}}) \in \llbracket c_1^{\text{abs}} \rrbracket. \end{aligned}$$

Proposition 2. $(\sigma_1^{\text{abs}}, c^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, \text{ret } a)$ iff. $(\sigma_1^{\text{abs}}, a, \sigma_2^{\text{abs}}) \in \llbracket c^{\text{abs}} \rrbracket$.

Contextual Triples. The proof contexts $S_{\forall}$ and $S_{\forall\exists}$ collect standard and relational function specifications, respectively. An environment χ is *valid* with respect to $S_{\forall}$, written $\text{Valid}(\chi, S_{\forall})$, if for every specification in $S_{\forall}$, any $\vec{a}$, any σ_1^{con} satisfying $P(\vec{a})$, and any $(r^{\text{con}}, \sigma_2^{\text{con}})$ such that $(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \chi(f)$, σ_2^{con} satisfies $Q(\vec{a}, r^{\text{con}})$.

We write $\text{RelValid}(\chi, S_{\forall\exists})$ when χ is *relationally valid* with respect to $S_{\forall\exists}$: for every relational specification in $S_{\forall\exists}$, any $\vec{a}$, any σ_1^{con} and σ_1^{abs} satisfying $P(\vec{a})$, and any $(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \chi(f)$, there exist r^{abs} and σ_2^{abs} such that $(\sigma_1^{\text{abs}}, r^{\text{abs}}, \sigma_2^{\text{abs}}) \in \llbracket c^{\text{abs}} \rrbracket$ and $(\sigma_2^{\text{con}}, \sigma_2^{\text{abs}})$ satisfies $Q(\vec{a}, r^{\text{con}}, r^{\text{abs}})$.

Definition 2 (Contextual Hoare Triple). For proof contexts $S_{\forall}$, $S_{\forall\exists}$ and a program $c^{\text{con}} \in \text{Imp}$,

$$S_{\forall}, S_{\forall\exists} \vdash \{P\} c^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\}$$

holds if, for any χ with $\text{Valid}(\chi, S_{\forall})$ and $\text{RelValid}(\chi, S_{\forall\exists})$, any σ_1^{con} satisfying P , and any $(\sigma_2^{\text{con}}, r^{\text{con}})$ such that $(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \llbracket c^{\text{con}} \rrbracket^{\chi}$, we have that σ_2^{con} satisfies $Q(r^{\text{con}})$.

Definition 3 (Relational Contextual Hoare Triple). For proof contexts $S_{\forall}$, $S_{\forall\exists}$ and a program $c^{\text{con}} \in \text{Imp}$,

$$S_{\forall}, S_{\forall\exists} \vdash \{P\} c^{\text{con}} \{r^{\text{con}}. Q(r^{\text{con}})\}$$

holds if, for any χ with $\text{Valid}(\chi, S_{\forall})$ and $\text{RelValid}(\chi, S_{\forall\exists})$, any σ_1^{con} , σ_1^{abs} , c^{abs} satisfying P , and any $(\sigma_1^{\text{con}}, r^{\text{con}}, \sigma_2^{\text{con}}) \in \llbracket c^{\text{con}} \rrbracket^{\chi}$, there exist c_2^{abs} and σ_2^{abs} such that $(\sigma_1^{\text{abs}}, c^{\text{abs}}) \hookrightarrow (\sigma_2^{\text{abs}}, c_2^{\text{abs}})$ and $(\sigma_2^{\text{con}}, \sigma_2^{\text{abs}}, c_2^{\text{abs}})$ satisfies $Q(r^{\text{con}})$.

Inference rules. Fig. 5 presents additional inference rules for hybrid reasoning. The SIM-BIND rule supports symmetric reasoning by aligning the concrete sequential composition $c_1^{\text{con}}; c_2^{\text{con}}$ with the abstract let-binding $\text{let } r = c^{\text{abs}} \text{ in } K(r)$. The first judgment connects c_1^{con} to the abstract computation c^{abs} that produces a result r ; the second relates c_2^{con} to the abstract continuation $K(r)$. The REL-CALL rule allows relational proofs for functions to be reused. This rule can be

$$\begin{array}{c}
\text{SIM-BIND} \\
\frac{S_{\forall}, S_{\forall\exists} \vdash \{[c^{\text{abs}}] \wedge \mathcal{P}\} \ c_1^{\text{con}} \ \{\exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge \mathcal{R}(r^{\text{abs}})\} \\
\quad S_{\forall}, S_{\forall\exists} \vdash \forall r^{\text{abs}}. \{[K(r^{\text{abs}})] \wedge \mathcal{R}(r^{\text{abs}})\} \ c_2^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}{S_{\forall}, S_{\forall\exists} \vdash \{[\text{let } r^{\text{abs}} = c^{\text{abs}} \text{ in } K(r^{\text{abs}})] \wedge \mathcal{P}\} \ c_1^{\text{con}}; c_2^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}} \\
\\
\text{REL-CALL} \\
\frac{(f : \Pi(\vec{a} : A). \{[c^{\text{abs}}] \wedge \mathcal{P}(\vec{a})\} \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge \mathcal{Q}(\vec{a}, r^{\text{con}}, r^{\text{abs}})\}) \in S_{\forall\exists}}{S_{\forall}, S_{\forall\exists} \vdash \{[c^{\text{abs}}] \wedge \mathcal{P}(\vec{a})\} \ \text{call } f \ \{r^{\text{con}}. \exists r^{\text{abs}}. [\text{ret } r^{\text{abs}}] \wedge \mathcal{Q}(\vec{a}, r^{\text{con}}, r^{\text{abs}})\}} \\
\\
\text{HOARE-CALL} \\
\frac{f : \Pi(\vec{a} : A). \{P(\vec{a})\} \{r^{\text{con}}. Q(\vec{a}, r^{\text{con}})\} \in S_{\forall}}{S_{\forall}, S_{\forall\exists} \vdash \{P(\vec{a})\} \ \text{call } f \ \{r^{\text{con}}. Q(\vec{a}, r^{\text{con}})\}} \\
\\
\text{ABS-CHOICE-L} \\
\frac{\{[c_1^{\text{abs}}] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}{\{[\text{choice}(c_1^{\text{abs}}, c_2^{\text{abs}})] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}} \\
\\
\text{ABS-CHOICE-R} \\
\frac{\{[c_2^{\text{abs}}] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}{\{[\text{choice}(c_1^{\text{abs}}, c_2^{\text{abs}})] \wedge \mathcal{P}\} \ c^{\text{con}} \ \{r^{\text{con}}. \mathcal{Q}(r^{\text{con}})\}}
\end{array}$$

Fig. 5: More Inference Rules for Hybrid Reasoning

used for symmetric reasoning for concrete functions by assigning different abstract programs to different function calls through rule SIM-BIND. The HOARE-CALL rule looks up a function's standard specification from $S_{\forall}$ and produces the corresponding standard Hoare triple. Combined with the CON-FOCUS rule, it enables independent evaluation of function calls within the concrete program. The ABS-CHOICE rules handle nondeterministic abstract programs: to refine $\text{choice}(c_1^{\text{abs}}, c_2^{\text{abs}})$, it suffices to refine either branch.

5 Case Study: Hybrid Sorting

In this section, we apply our approach to a recursive implementation of a hybrid sorting algorithm depicted in Fig. 2. We explain the overall proof strategy and demonstrate how relational and standard Hoare logics can be combined to efficiently verify implementation correctness.

The concrete program takes a singly linked list, and if the list contains fewer than 8 elements, it employs an insertion sort. For input lists with 8 or more elements, this program proceeds with a merge sort approach. In contrast, the abstract program nondeterministically selects an external sorting, which is agnostic regarding the specific sorting mechanics applied, and merge sort. Following our approach, we formalize the implementation correctness using the following

relational Hoare triple:

$$\begin{array}{c} \{[\text{mergesort}_A(l_x)] \wedge \text{storesll}(x, l_x)\} \\ \text{mergesort}(x) \\ \{r^{\text{con}}. \exists l_r. [\text{ret } l_r] \wedge \text{storesll}(r^{\text{con}}, l_r)\} \end{array} \quad (4)$$

Here, $\text{storesll}(x, l_x)$ asserts that variable x stores a singly linked list l_x . This triple states that if the abstract program is about to sort the same list l_x stored at x , then once the concrete program terminates with return value r^{con} , there exists a list l_r stored at r^{con} such that the abstract program can produce exactly l_r .

To prove this relational Hoare triple, we first exploit a clear alignment between the concrete and abstract programs. In particular, the abstract program is structured as a nondeterministic choice. We must angelically select the branch in the abstract program that aligns with the if-branching of the concrete program. Specifically, when the input list l_x has a length of at least 8, we can apply rule ABS-CHOICE-R to let the abstract program angelically choose the merge sort branch to mirror the concrete merge sort steps. More concretely, when the concrete statement $\text{split}(x, \&p, \&q)$ splits the input list, the abstract program $\text{split}_A(l_x, l_p, l_q)$ can likewise simulate this splitting. When the concrete statement $\text{mergesort}(p)$ performs a merge sort on the split sublist, the abstract program $\text{mergesort}_A(l_p)$ can simulate the merge sort. Finally, when the concrete statement $\text{merge}(p, q)$ merges the respective sub-results, the abstract program $\text{merge}_A(l_p, l_q)$ can simulate this same merge operation. Then, we can apply rules SIM-BIND and REL-CALL to decompose the proof into several small refinements, such as proving that $\text{split}(x, \&p, \&q)$ refines $\text{split}_A(l_x, l_p, l_q)$:

$$\begin{array}{c} \{[\text{split}_A(l_x, l_p, l_q)] \wedge \text{storesll}(x, l_x) * \text{storesll}(p, l_p) * \text{storesll}(q, l_q)\} \\ \text{split}(x, \&p, \&q) \\ \{\exists l'_x, l'_p, l'_q. [\text{ret } (l'_x, l'_p, l'_q)] \wedge \text{storesll}(x, l'_x) * \text{storesll}(p, l'_p) * \text{storesll}(q, l'_q)\} \end{array}$$

Next, we consider the scenario when the input list l_x is shorter than eight elements, in which case the concrete program selects insertion sort. For this case, we apply rule ABS-CHOICE-L to guide the abstract program to employ the external sort. Because this external sort behaves as a black box (any sorted result is valid), we independently evaluate the abstract program and the concrete program. For the abstract program, we apply rule ABS-PURE to reduce $\text{extsort}_A(l_x)$ to $\text{ret } l_r$ where l_r is a sorted permutation of l_x . For the concrete program, we apply rules CON-FOCUS and HOARE-CALL to evaluate the call statement $\text{inssort}(x)$. This reduces to proving the insertion sort's functional correctness as described by the following Hoare triple:

$$\{\text{storesll}(x, l_x)\} \text{inssort}(x) \{r^{\text{con}}. \text{permutation}(l_x, l_r) \wedge \text{sorted}(l_r) \wedge \text{storesll}(r^{\text{con}}, l_r)\}$$

This demonstrates how the hybrid approach delegates part of the relational proof to standard Hoare logic when the concrete program has no abstract counterpart.

Having verified the implementation, we must now establish the algorithm correctness of the abstract program and then compose the two proofs. The specification for the abstract program is:

$$\{\text{True}\} \text{mergesort}_A(l_x) \{l_r. \text{permutation}(l_x, l_r) \wedge \text{sorted}(l_r)\}$$

Composing this with implementation correctness via rule VERTICAL-COMP, we derive the functional correctness of the concrete program:

$$\begin{aligned} & \{\text{storesll}(x, l_x)\} \\ & \quad \text{mergesort}(x) \\ & \{r^{\text{con}}. \exists l_r. \text{permutation}(l_x, l_r) \wedge \text{sorted}(l_r) \wedge \text{storesll}(r^{\text{con}}, l_r)\} \end{aligned}$$

In summary, this example illustrates how our hybrid reasoning approach can flexibly employ relational Hoare logic and standard Hoare logic to prove that the concrete program implements the abstract specification for complex algorithms.

6 Related Work

Here, we discuss related work, including approaches for verifying functional correctness and frameworks for relational Hoare logic.

Functional correctness for complex programs. Wang et al. [40] developed a graph library in Rocq. Subsequently, Mohan et al. [31] extended this library to support undirected graphs. Their line of work directly proves the functional correctness of C programs that manipulate graph data structures using standard Hoare logic. In contrast, this paper proves the functional correctness of imperative programs through implementation correctness and algorithm correctness. Precisely, our approach proves that a recursive DFS implementation refines an iterative abstract description, and the algorithm correctness proof can be reused for different concrete implementations. During the verification, our work builds upon their fundamental definitions of graphs, enabling the algorithm correctness proof to leverage existing theorems in graph theory. Cao et al. [10] proved the functional correctness of an imperative insertion function for BSTs with the two-layer method [3]. Meanwhile, LAProof [22] offers a specialized library for verifying linear algebra programs, featuring case studies such as matrix multiplication. However, they both employ standard (separation) Hoare logic to establish the implementation correctness. In this paper, we emphasize that such an implementation correctness proof is essentially relational since it connects the abstract specification of tree insertion with its concrete imperative counterpart. We thus prove correctness via relational Hoare logic, offering a more intuitive reasoning pattern.

Building on the Isabelle Refinement Framework [25], Lammich and Neumann [24] proposed a layered method to prove the functional correctness of DFS-based abstract functional algorithms. With Imperative HOL [9], this framework enables one to first verify the correctness of algorithm descriptions, which are purely

functional, and then refine that abstract description step by step. The final result is an executable imperative program. While we both rely on refinement to prove implementation correctness, we employ relational Hoare triples and support abstract programs with nondeterminism and state.

Relational Hoare logic frameworks. Relational Hoare logic, introduced by Benton [7], extends traditional Hoare logic to relate two programs. Many relational verification frameworks have been proposed to verify relational properties, including equivalence [43,23,41,30,4] and refinement [5,12]. This paper focuses on verifying program refinement, which can be formally captured via simulation relations. For concurrent program verification, Liang and Feng [26] and Turon et al. [37] proposed program-as-resource assertions and relational Hoare triples based on powerful ghost states [39]. Follow-up research includes a series of works [29,27,28] and another series [34,14,33] based on the Iris framework [21,20]. Among existing relational logics, ReLoC [15] is especially closely related to our work in its support for refinement reasoning. Although program-as-resource assertions were originally designed for concurrent settings, we adopt them for verifying sequential programs. This specialization enables several advantages. First, it allows us to use denotational semantics for sequential programs, avoiding the complexity of step-indexed models. Second, program-as-resource assertions allow us to leverage powerful reasoning principles, such as the REL-WH rule, to address nontrivial refinements between structurally dissimilar programs. At the same time, general frameworks like Iris provide benefits that are orthogonal to our goals. For example, Timany et al. [35] proposed a well-bracketed Hoare logic based on Iris, for reasoning about well-bracketed sequential programs, where each function call must return before its caller can resume execution.

Aguirre et al. proposed a unary higher-order logic (UHOL) and a relational High-order logic (RHOL) [1] for reasoning about relational properties of a simply typed λ -calculus. RHOL provides inference rule [UHOL-L] to allow RHOL proofs to fall back to UHOL reasoning. This rule serves a role similar to the focusing rule CON-FOCUS in this paper, enabling independent evaluation on one side. While RHOL inherits the type-directed structure of relational refinement type systems and aims to support more automation, this paper emphasizes focusing and alignment rules (e.g., CON-FOCUS, SIM-BIND) that allow provers to flexibly switch between relational and standard Hoare reasoning within a single proof.

The work of Gladshtein et al. [16] focuses on verifying the correctness of operations where identical or sparse data are stored in arrays through compression or other space-efficient representations. Such implementations often involve multiple loops with intricate invariants. The authors argue that relational program logics are well-suited to address these verification tasks, an assessment that aligns with our own perspective. Relational Hoare logic can offer an intuitive and convenient framework for proving implementation correctness. However, they introduce the framework LGTM (Logic for Graceful Tensor Manipulation), which dynamically determines the number k of traces to compare, allowing k to depend on program variables, while we rely on relational Hoare triples.

Dickerson et al. [13] propose RHLE, a logic dedicated to verifying $\forall\exists$ relational properties. A central feature of their framework is the proof context $S_\exists$ which leverages function specifications and ghost variables to ensure that abstract programs achieve some existential behaviors. By combining $S_\exists$ with the standard proof context $S_\forall$ for concrete programs, they can derive $\forall\exists$ Hoare judgments. In contrast, our work does not define a $\exists$ proof context for abstract programs. We employ the relational proof context $S_{\forall\exists}$ that relates the concrete program’s functions to abstract programs.

Several works have proposed automated approaches for relational verification. Unno et al. [38] present a constraint-based approach to relational verification that supports a variety of relational properties, including refinement. Beutner and Finkbeiner [8] propose an automated method for verifying $\forall^*\exists^*$ hyperproperties, and Itzhaky et al. [19] reduce hyperproperty verification to the satisfiability of constrained Horn clauses (CHCs). In this paper, we concentrate on interactive verification of refinement properties. Nonetheless, techniques developed in those works may complement our method in future integration efforts.

7 Conclusion and Future Work

In this paper, we propose a hybrid reasoning method that combines relational and standard Hoare logic for verifying sequential programs. We show that employing relational Hoare logic for implementation correctness is advantageous: it enables more intuitive loop invariants and allows us to leverage algorithm correctness to reduce proof complexity during implementation verification. A key feature of our approach is the VERTICAL-COMP rule, which supports modular verification by cleanly separating verification. Implementation correctness is established relationally, while algorithm correctness is proven once using standard Hoare logic. This compositional structure significantly reduces proof duplication and facilitates reuse across implementations.

In this work, we focus on partial correctness; that is, we do not address reasoning about nontermination. One possible indirect approach to handling total correctness is to enrich the abstract program state with time credits and prove that the program must terminate within the given bounds. If the time budget reaches zero, the program produces an error. While this technique can be encoded in our framework, a more principled and general treatment of nontermination remains an important direction for future work. Another promising direction is to automate the verification of implementation correctness, for instance, through novel tactics and symbolic execution. Such automation can reduce the proof burden and make our approach more accessible for large-scale verification projects. Besides, we aim to support richer program features, such as function pointers, to further demonstrate both the flexibility and scalability of the method. Ultimately, we hope that our hybrid approach will serve as a robust foundation for broadly applicable verification frameworks.

References

1. Aguirre, A., Barthe, G., Gaboardi, M., Garg, D., Strub, P.: A relational logic for higher-order programs. *Proc. ACM Program. Lang.* **1**(ICFP), 21:1–21:29 (2017). <https://doi.org/10.1145/3110265>
2. Appel, A.W.: Verified Functional Algorithms, vol. 3 (2020), <https://softwarefoundations.cis.upenn.edu/vfa-current/index.html>
3. Appel, A.W.: Coq’s vibrant ecosystem for verification engineering (invited talk). In: Popescu, A., Zdancewic, S. (eds.) *CPP ’22: 11th ACM SIGPLAN International Conference on Certified Programs and Proofs*, Philadelphia, PA, USA, January 17 – 18, 2022. pp. 2–11. ACM (2022). <https://doi.org/10.1145/3497775.3503951>
4. Banerjee, A., Nagasamudram, R., Naumann, D.A., Nikouei, M.: A relational program logic with data abstraction and dynamic framing. *ACM Trans. Program. Lang. Syst.* **44**(4), 25:1–25:136 (2022). <https://doi.org/10.1145/3551497>
5. Barthe, G., Crespo, J.M., Kunz, C.: Beyond 2-safety: Asymmetric product programs for relational program verification. In: Artëmov, S.N., Nerode, A. (eds.) *Logical Foundations of Computer Science, International Symposium, LICS 2013*, San Diego, CA, USA, January 6–8, 2013. *Proceedings. Lecture Notes in Computer Science*, vol. 7734, pp. 29–43. Springer (2013). https://doi.org/10.1007/978-3-642-35722-0_3
6. Beckert, B., Ulbrich, M.: Trends in relational program verification. In: Müller, P., Schaefer, I. (eds.) *Principled Software Development - Essays Dedicated to Arnd Poetzsch-Heffter on the Occasion of his 60th Birthday*. pp. 41–58. Springer (2018). https://doi.org/10.1007/978-3-319-98047-8_3
7. Benton, N.: Simple relational correctness proofs for static analyses and program transformations. In: Jones, N.D., Leroy, X. (eds.) *Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2004*, Venice, Italy, January 14–16, 2004. pp. 14–25. ACM (2004). <https://doi.org/10.1145/964001.964003>
8. Beutner, R., Finkbeiner, B.: Software verification of hyperproperties beyond k-safety. In: Shoham, S., Vizel, Y. (eds.) *Computer Aided Verification - 34th International Conference, CAV 2022*, Haifa, Israel, August 7–10, 2022, *Proceedings, Part I*. *Lecture Notes in Computer Science*, vol. 13371, pp. 341–362. Springer (2022). https://doi.org/10.1007/978-3-031-13185-1_17
9. Bulwahn, L., Krauss, A., Haftmann, F., Erkök, L., Matthews, J.: Imperative functional programming with isabelle/hol. In: Mohamed, O.A., Muñoz, C.A., Tahar, S. (eds.) *Theorem Proving in Higher Order Logics, 21st International Conference, TPHOLs 2008*, Montreal, Canada, August 18–21, 2008. *Proceedings. Lecture Notes in Computer Science*, vol. 5170, pp. 134–149. Springer (2008). https://doi.org/10.1007/978-3-540-71067-7_14
10. Cao, Q., Wang, S., Hobor, A., Appel, A.W.: Proof pearl: Magic wand as frame. *arXiv preprint arXiv:1909.08789* (2019)
11. Chen, R., Cohen, C., Levy, J.J., Merz, S., Thery, L.: Formal proofs of tarjan’s algorithm in why3, coq, and isabelle (2018)
12. Clochard, M., Marché, C., Paskevich, A.: Deductive verification with ghost monitors. *Proc. ACM Program. Lang.* **4**(POPL), 2:1–2:26 (2020). <https://doi.org/10.1145/3371070>
13. Dickerson, R., Ye, Q., Zhang, M.K., Delaware, B.: RHLE: modular deductive verification of relational $\forall \exists$ properties. In: Sergey, I. (ed.) *Programming Languages*

- and Systems - 20th Asian Symposium, APLAS 2022, Auckland, New Zealand, December 5, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13658, pp. 67–87. Springer (2022). https://doi.org/10.1007/978-3-031-21037-2_4
14. Frumin, D., Krebbers, R., Birkedal, L.: Reloc: A mechanised relational logic for fine-grained concurrency. In: Dawar, A., Grädel, E. (eds.) Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018. pp. 442–451. ACM (2018). <https://doi.org/10.1145/3209108.3209174>
 15. Frumin, D., Krebbers, R., Birkedal, L.: Reloc reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. CoRR **abs/2006.13635** (2020), <https://arxiv.org/abs/2006.13635>
 16. Gladshstein, V., Zhao, Q., Ahrens, W., Amarasinghe, S.P., Sergey, I.: Mechanised hypersafety proofs about structured data. Proc. ACM Program. Lang. **8**(PLDI), 647–670 (2024). <https://doi.org/10.1145/3656403>
 17. Hoare, C.A.R.: An axiomatic basis for computer programming. Commun. ACM **12**(10), 576–580 (oct 1969). <https://doi.org/10.1145/363235.363259>
 18. Huet, G.: The zipper. Journal of Functional Programming **7**(5), 549–554 (1997). <https://doi.org/10.1017/S0956796897002864>
 19. Itzhaky, S., Shoham, S., Vizel, Y.: Hyperproperty verification as CHC satisfiability. In: Weirich, S. (ed.) Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14577, pp. 212–241. Springer (2024). https://doi.org/10.1007/978-3-031-57267-8_9
 20. Jung, R., Krebbers, R., Jourdan, J., Bizjak, A., Birkedal, L., Dreyer, D.: Iris from the ground up: A modular foundation for higher-order concurrent separation logic. J. Funct. Program. **28**, e20 (2018). <https://doi.org/10.1017/S0956796818000151>
 21. Jung, R., Swasey, D., Sieczkowski, F., Svendsen, K., Turon, A., Birkedal, L., Dreyer, D.: Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In: Rajamani, S.K., Walker, D. (eds.) Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015. pp. 637–650. ACM (2015). <https://doi.org/10.1145/2676726.2676980>
 22. Kellison, A.E., Appel, A.W., Tekriwal, M., Bindel, D.: Laproof: A library of formal proofs of accuracy and correctness for linear algebra programs. In: 2023 IEEE 30th Symposium on Computer Arithmetic (ARITH). pp. 36–43 (2023). <https://doi.org/10.1109/ARITH58626.2023.00021>
 23. Kundu, S., Tatlock, Z., Lerner, S.: Proving optimizations correct using parameterized program equivalence. In: Hind, M., Diwan, A. (eds.) Proceedings of the 2009 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2009, Dublin, Ireland, June 15-21, 2009. pp. 327–337. ACM (2009). <https://doi.org/10.1145/1542476.1542513>
 24. Lammich, P., Neumann, R.: A framework for verifying depth-first search algorithms. In: Proceedings of the 2015 Conference on Certified Programs and Proofs. pp. 137–146 (2015)
 25. Lammich, P., Tuerk, T.: Applying data refinement for monadic programs to hopcroft’s algorithm. In: International Conference on Interactive Theorem Proving. pp. 166–182. Springer (2012)

26. Liang, H., Feng, X.: Modular verification of linearizability with non-fixed linearization points. In: Boehm, H., Flanagan, C. (eds.) ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16–19, 2013. pp. 459–470. ACM (2013). <https://doi.org/10.1145/2491956.2462189>
27. Liang, H., Feng, X.: A program logic for concurrent objects under fair scheduling. In: Bodik, R., Majumdar, R. (eds.) Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016. pp. 385–399. ACM (2016). <https://doi.org/10.1145/2837614.2837635>
28. Liang, H., Feng, X.: Progress of concurrent objects with partial methods. *Proc. ACM Program. Lang.* **2**(POPL), 20:1–20:31 (2018). <https://doi.org/10.1145/3158108>
29. Liang, H., Feng, X., Shao, Z.: Compositional verification of termination-preserving refinement of concurrent programs. In: Henzinger, T.A., Miller, D. (eds.) Joint Meeting of the Twenty-Third EACSL Annual Conference on Computer Science Logic (CSL) and the Twenty-Ninth Annual ACM/IEEE Symposium on Logic in Computer Science (LICS), CSL-LICS '14, Vienna, Austria, July 14 - 18, 2014. pp. 65:1–65:10. ACM (2014). <https://doi.org/10.1145/2603088.2603123>
30. Maillard, K., Hritcu, C., Rivas, E., Muylder, A.V.: The next 700 relational program logics. *Proc. ACM Program. Lang.* **4**(POPL), 4:1–4:33 (2020). <https://doi.org/10.1145/3371072>
31. Mohan, A., Leow, W.X., Hobor, A.: Functional correctness of C implementations of dijkstra's, kruskal's, and prim's algorithms. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12760, pp. 801–826. Springer (2021). https://doi.org/10.1007/978-3-030-81688-9_37
32. Reynolds, J.C.: Separation logic: A logic for shared mutable data structures. In: 17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22–25 July 2002, Copenhagen, Denmark, Proceedings. pp. 55–74. IEEE Computer Society (2002). <https://doi.org/10.1109/LICS.2002.1029817>
33. Spies, S., Gäher, L., Gratzner, D., Tassarotti, J., Krebbers, R., Dreyer, D., Birkedal, L.: Transfinite iris: resolving an existential dilemma of step-indexed separation logic. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. p. 80–95. PLDI 2021, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3453483.3454031>
34. Tassarotti, J., Jung, R., Harper, R.: A higher-order logic for concurrent termination-preserving refinement. In: Yang, H. (ed.) Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings. Lecture Notes in Computer Science, vol. 10201, pp. 909–936. Springer (2017). https://doi.org/10.1007/978-3-662-54434-1_34
35. Timany, A., Guéneau, A., Birkedal, L.: The logical essence of well-bracketed control flow. *Proc. ACM Program. Lang.* **8**(POPL), 575–603 (2024). <https://doi.org/10.1145/3632862>
36. Turon, A., Dreyer, D., Birkedal, L.: Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In: Proceedings of the 18th ACM

- SIGPLAN International Conference on Functional Programming. p. 377–390. ICFP '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2500365.2500600>
37. Turon, A.J., Thamsborg, J., Ahmed, A., Birkedal, L., Dreyer, D.: Logical relations for fine-grained concurrency. In: Giacobazzi, R., Cousot, R. (eds.) The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013. pp. 343–356. ACM (2013). <https://doi.org/10.1145/2429069.2429111>
 38. Unno, H., Terauchi, T., Koskinen, E.: Constraint-based relational verification. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12759, pp. 742–766. Springer (2021). https://doi.org/10.1007/978-3-030-81685-8_35
 39. Vafeiadis, V.: Modular fine-grained concurrency verification. Ph.D. thesis, University of Cambridge, UK (2008), <https://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.612221>
 40. Wang, S., Cao, Q., Mohan, A., Hobor, A.: Certifying graph-manipulating C programs via localizations within data structures. Proc. ACM Program. Lang. **3**(OOPSLA), 171:1–171:30 (2019). <https://doi.org/10.1145/3360597>
 41. Wang, Y., Dillig, I., Lahiri, S.K., Cook, W.R.: Verifying equivalence of database-driven applications. Proc. ACM Program. Lang. **2**(POPL), 56:1–56:29 (2018). <https://doi.org/10.1145/3158144>
 42. Wu, S., Wu, X., Yang, C., Cao, Q.: Intuitive verification of sequential programs using hybrid reasoning: Case studies (2026). <https://doi.org/10.5281/zenodo.19787359>
 43. Yang, H.: Relational separation logic. Theor. Comput. Sci. **375**(1-3), 308–334 (2007). <https://doi.org/10.1016/J.TCS.2006.12.036>